
SPRINT 3

ANÁLISE DOS RESULTADOS

Aluna: Ana Clara Viganó P. de Oliveira - 412887 **Aluna:** Jasmine Masson Alves - 427466

Análise dos Resultados dos Mecanismos de Atenção

1. Self-Attention simples

Mecanismo onde tokens comparam diretamente seus vetores de embedding originais usando produto escalar (*dot product*), normalizando as pontuações pelo *Softmax* para calcular uma soma (vetor de contexto). Seu nome deve-se a não utilização de pesos para o cálculo.

Análise dos Códigos e Resultados (`self.attention.simples.py`, `self.attention.generalizado.py`)

- A comparação entre pares de vetores de entrada foi implementada de duas formas:
 - Em laço manual via `torch.dot`.
 - Em forma matricial vetorizada através de `inputs @ inputs.T` (matriz 6×6). Isso reduz o tempo de processamento.
- **Normalização:** A aplicação do `torch.softmax()` permite que a soma das probabilidades de cada linha da matriz de atenção seja igual a 1.0.
- **Vetor de Contexto:** A operação `attn_weights @ inputs` combina a matriz 6×6 com a entrada 6×3 , o que gera vetores de contexto "enriquecidos" de dimensão 6×3 .

2. Scaled Dot-Product Attention

Evolução da self-attention que projeta cada embedding de entrada em três vetores distintos: **Query** (Q), **Key** (K) e **Value** (V). Aplica um fator de escala ($\frac{1}{\sqrt{d_k}}$) antes da função *Softmax* para evitar que o produto atinja valores altos, o que previne a saturação de gradientes.

Importante lembrar que diz-se "evolução do self-attention" pois utilizamos pesos treináveis para calcular a atenção.

Análise dos Códigos e Resultados (scale_dot_product_attention.py, self_attention_class.py, exercicio.py)

- **Evolução das Implementações (v1 X v2):**

- SelfAttention_v1: Declara explicitamente matrizes com `nn.Parameter(torch.rand(d_in, d_out))`.
- SelfAttention_v2: Substitui as matrizes manuais por `nn.Linear(d_in, d_out, bias=False)`, que realiza a inicialização adequada de pesos para o treinamento.

- **Transposição de Pesos no exercicio.py:** O script provou a equivalência entre as duas abordagens. Como a camada `nn.Linear` do PyTorch armazena os pesos na forma (d_{out}, d_{in}) , foi necessário transpor a matriz para copiá-los para a v1 (d_{in}, d_{out}) .

- **Resultado:** A `torch.allclose(context_v1, context_v2)` retornou True, confirmando a equivalência dos cálculos matemáticos.

3. Causal Attention

Garante a propriedade autoregressiva dos modelos de linguagem, impede que o token atual visualize posições futuras da sequência.

Análise dos Códigos e Resultados (causal_attention_class.py, causal_attention.py)

- **Criação e Registro da Máscara:** É criada uma matriz triangular superior com `torch.triu(..., diagonal=1)` e registrada no módulo usando `self.register_buffer('mask', ...)`. O uso de `register_buffer` garante que a máscara acompanhe o dispositivo (CPU/GPU) sem ser tratada como parâmetro treinável.
- **Funcionamento e Papel da Máscara:** A máscara isola as posições acima da diagonal principal da matriz de atenção. Sua função essencial é restringir o campo de visão do modelo, a máscara bloqueia todas as posições futuras na sequência, permitindo que cada posição interaja unicamente com os tokens passados e com ela mesma.
- **Prevenção de Vazamento (*Information Leakage*):** O código aplica `.masked_fill_(mask.bool(), -torch.inf)` sobre os scores de atenção. Ao aplicar o *Softmax*, o valor $-\infty$ torna-se 0.0, anulando o peso de atenção dos tokens futuros e renormalizando a linha apenas com os tokens passados e o atual.

4. Regularização de Atenção por Dropout

Técnica de regularização que desativa aleatoriamente valores na matriz de atenção durante o treinamento, o que reduz o *overfitting*.

Análise dos Códigos e Resultados (dropout.py)

- **Compensação de Escala:** Ao aplicar `torch.nn.Dropout(p=0.5)` em uma matriz de valor 1.0, metade das células tornam-se 0.0, enquanto os elementos remanescentes são multiplicados por $\frac{1}{1-p} = 2.0$.

- **Resultado:** Garante que a soma esperada dos pesos mantidos permaneça consistente com a distribuição original, isso preserva o equilíbrio matemático nos gradientes.

5. Multi-Head Attention (MHA)

Divide a dimensão de saída d_{out} em h subespaços menores ($\text{head_dim} = d_{\text{out}}/\text{num_heads}$), isso permite que o modelo preste atenção em diferentes aspectos da sequência ao mesmo tempo.

Análise dos Códigos e Resultados (`multi_head_attention_class.py`, `view_transpose.py`, `GPT_2_modulo_attention.py`)

- **Wrapper vs. Paralelo:**
 - `MultiHeadAttentionWrapper`: Instancia múltiplos objetos `CausalAttention` dentro de um `nn.ModuleList` e concatena suas saídas. Ineficiente do ponto de vista de GPU mas, pe didático para entender multi heads.
 - `MultiHeadAttention`: Faz Q, K, V em uma única multiplicação matricial e reorganiza os tensores com `.view()` e `.transpose(1, 2)`, executando todas as cabeças em paralelo.
- **Instanciação com Especificações do GPT-2:** O script `GPT_2_modulo_attention.py` configurou a classe com $d_{\text{in}} = 768$, $d_{\text{out}} = 768$, $\text{num_heads} = 12$ e $\text{context_length} = 1024$. Internamente, cada cabeça processa vetores com $\text{head_dim} = 768/12 = 64$ dimensões.

6. Experimentos e Visualização de Matrizes

Análise do comportamento do mecanismo MHA alternando a quantidade de cabeças e verificando os mapas/gráficos (*heatmaps*).

Análise dos Códigos e Resultados (`single_vs_multi_head.py`, `matrizes_attention.py`)

- **Impacto da Divisão em Cabeças** (`single_vs_multi_head.py`):
 - Ao processar o mesmo batch $1 \times 8 \times 8$ alterando o parâmetro `num_heads` (1, 2 e 4), as dimensões do tensor de saída mantêm-se constantes em (1, 8, 8).
 - Os valores numéricos gerados mudam bastante. A diferença média entre 1 e 2 heads, assim como entre 2 e 4 heads, comprova que projetar os dados em múltiplos subespaços de dimensão reduzida altera a combinação das informações de contexto.
- **Análise Gráfica** (`matrizes_attention.py`):
 - As plotagens mostram a estrutura da máscara causal. A primeira linha atribui 1.0 de atenção ao próprio Token 1.
 - Cada cabeça apresenta uma distribuição probabilística única ao longo do triângulo, confirmando as pequenas variações de atenção aprendidas em cada subespaço em paralelo.

Conclusão

A implementação e análise prática dos mecanismos de atenção mostram a evolução conceitual e técnica que é a base da arquitetura Transformer e os modelos de linguagem (LLMs) modernos.

Da atenção simples para a autoatenção escalada com pesos treináveis (W_q, W_k, W_v) mostra como o modelo ganha flexibilidade ao projetar os embeddings de entrada em três subespaços diferentes (Query, Key e Value). A máscara causal assegura a propriedade autoregressiva na geração de texto, garante que o cálculo de contexto dependa apenas dos tokens passados e do token atual, eliminando qualquer vazamento de informação do futuro.

Ademais, a consolidação do *Multi-Head Attention* (MHA) revelou a eficiência de dividir a dimensão de representação em múltiplas heads independentes. Isso permite que a rede capture diferentes padrões contextuais em paralelo sem aumentar o tamanho final dos tensores.

1 Referências Bibliográficas

RASCHKA, Sebastian. Build a large language model (from scratch). Simon and Schuster, 2024.